

Relatorio Completo - IDE Pascal e Analisador Lexico

Relatorio Tecnico Completo

IDE Pascal e Analisador Lexico em Java

Documentacao do codigo-fonte da pasta IDE-PASCAL

Arquivos estudados: Clone.java, Lexer.java e Token.java

Linguagem: Java | Interface: Java Swing | Tema: Analise Lexica de Mini Pascal

Gerado para estudo do funcionamento, metodos e implementacao do projeto.

1. Visao geral do projeto

O projeto implementa uma pequena IDE para escrever codigo inspirado em Pascal e executar uma analise lexica.

A analise lexica e a primeira etapa de um compilador: ela le o texto do programa, divide esse texto em partes menores chamadas tokens e classifica cada parte.

A aplicacao possui editor de codigo, numeracao de linhas, menus de arquivo, botoes de acao, destaque de sintaxe, tabela de tokens, tabela de erros, exportacao de resultado e uma funcao opcional de sugestao com IA local usando Ollama.

Ideia central: o usuario escreve codigo Pascal no editor. Ao clicar em analisar, a classe

Clone envia o texto para Lexer. O Lexer percorre o texto e cria objetos Token. A interface mostra os tokens e os erros encontrados.

2. Organizacao dos arquivos

| Arquivo | Responsabilidade

| Token.java

| Modelo de dados que representa cada token reconhecido pelo analisador.

| Lexer.java

| Motor da analise lexica. Le o codigo caractere por caractere, reconhece padroes e registra erros.

| Clone.java

| Interface grafica da IDE. Contem editor, menus, tabelas, temas, acoes de arquivo e integracao com o lexer.

| Documentario_Analisador_Lexico.html

| Documento explicativo anterior do projeto. Este novo relatorio aprofunda a leitura do codigo.

3. Como o sistema funciona

- O usuario digita ou abre um arquivo Pascal no editor.
- A interface monitora mudancas para atualizar o titulo, status e destaque visual.
- Quando a acao acaoAnalisar() e executada, o texto do editor e passado ao construtor de Lexer.
- O metodo analisar() do Lexer percorre o codigo e decide qual metodo de reconhecimento chamar.
- Os tokens criados sao armazenados em uma lista de Token.
- Os erros, como simbolos invalidos ou comentarios nao fechados, sao guardados em uma lista de mensagens.
- A classe Clone atualiza as tabelas de tokens e erros para o usuario estudar o resultado.

4. Classe Token.java

A classe Token e simples, mas fundamental. Ela guarda as informacoes produzidas pelo analisador lexico. Cada token possui lexema, classe, padrao, descricao e linha.

| Atributo | Significado

| lexema | Texto real encontrado no codigo, por exemplo program, idade ou :=.

| classe | Categoria do token, como PALAVRA_RESERVADA, IDENTIFICADOR ou NUMERO_INTEIRO.

| padrao | Regra ou forma geral que descreve o token reconhecido.

| descricao | Explicacao amigavel para exibir na interface e no relatorio exportado.

| linha | Numero da linha onde o token foi encontrado.

Metodos de Token

| Metodo | O que faz

| Token(String lexema, String classe, int linha) | Construtor reduzido. Cria um token sem padrao e sem descricao.

| Token(String lexema, String classe, String padrao, int linha) | Construtor intermediario.
Cria um token com padrao, mas sem descricao.

| Token(String lexema, String classe, String padrao, String descricao, int linha) | Construtor completo. Inicializa todos os campos do token.

| getLexema(), getClasse(), getPadrao(), getDescricao(), getLinha() | Metodos de acesso usados pela interface para preencher tabelas e relatorios.

| toString() | Retorna uma representacao textual curta do token, util para debug.

5. Classe Lexer.java

A classe Lexer contem o analisador lexico propriamente dito. Ela recebe uma string com o codigo inteiro e usa dois contadores principais: pos, que indica a posicao atual no texto, e linha, que indica a linha atual. Tambem possui duas listas: tokens e erros.

Palavras reservadas

O conjunto PALAVRAS_RESERVADAS lista termos da linguagem Pascal que nao devem ser classificados como nomes comuns. Exemplos: program, var, begin, end, if, while, function e procedure.

Fluxo principal da analise

```
while (pos < codigo.length()) {  
    char atual = codigo.charAt(pos);  
  
    if (Character.isWhitespace(atual)) consumirEspaco();  
    else if (atual == '{') reconhecerComentarioChaves();  
    else if (atual == '(' && proximoEh('*')) reconhecerComentarioParenteses();  
    else if (atual == '/' && proximoEh('/')) reconhecerComentarioLinha();  
    else if (Character.isLetter(atual)) reconhecerIdentificadorOuReservada();  
    else if (Character.isDigit(atual)) reconhecerNumero();  
    else if (atual == '"') reconhecerString();  
    else reconhecerSimbolo();  
}
```

Metodos de Lexer

| Metodo | O que faz | Como foi implementado

| Lexer(String codigo)

| Recebe o codigo que sera analisado.

| Guarda a string no atributo codigo. A analise real so comeca em analisar().

| analisar()

| Executa a analise lexica completa.

| Limpa listas anteriores, reinicia pos e linha, percorre todo o texto e chama metodos especificos conforme o caractere atual. No fim adiciona o token EOF.

| consumirEspaco()

| Ignora espacos, tabs e quebras de linha.

| Se encontrar \n, incrementa o contador de linha. Depois avanca uma posicao.

| reconhecerIdentificadorOuReservada()

| Reconhece palavras e nomes definidos pelo programador.

| Monta um texto enquanto houver letra, digito ou sublinhado. Depois compara a versao minuscula com PALAVRAS_RESERVADAS.

| reconhecerNumero()

| Reconhece numeros inteiros e reais.

| Primeiro consome digitos. Se encontrar ponto seguido de digito e nao for intervalo ..., continua consumindo a parte decimal e classifica como real.

| reconhecerString()

| Reconhece textos delimitados por aspas simples.

| Comeca na aspa inicial, acumula caracteres ate a aspa final e registra erro se chegar em quebra de linha ou fim do codigo sem fechar.

| reconhecerComentarioChaves()

| Reconhece comentarios no formato { ... }.

| Consome caracteres ate encontrar }. Atualiza linha em quebras de linha. Se nao fechar, registra erro.

| reconhecerComentarioParenteses()

| Reconhece comentarios no formato (* ... *).

| Avanca duas posicoes para pular (* , procura a sequencia *) e registra erro se nao encontrar.

| reconhecerComentarioLinha()

| Reconhece comentarios iniciados por //.

| Consome caracteres ate a quebra de linha. Esse comentario termina automaticamente no fim da linha.

| reconhecerSimbolo()

| Reconhece operadores e separadores.

| Usa switch para tratar +, -, *, /, =, <, >, :, ., virgula, ponto e virgula, parenteses e colchetes.

| adicionarSimbolo()

| Adiciona token de simbolo simples.

| Cria um Token com linha atual e avanca pos em uma unidade.

- | adicionarSimboloDuplo()
 - | Adiciona token composto por dois caracteres.
 - | Cria um Token e avança pos em duas unidades. Usado para <=, <>, >=, := e ...

- | proximoEh(char esperado)
 - | Verifica o próximo caractere sem avançar a leitura.
 - | Confere se existe posição seguinte e se ela contém o caractere esperado.

- | getErros()
 - | Retorna a lista de erros encontrados.
 - | Devolve uma nova lista para evitar alteração direta da lista interna.

- | isPalavraReservada(String palavra)
 - | Informa se uma palavra pertence ao conjunto reservado.
 - | Converte para minúsculas e consulta o conjunto PALAVRAS_RESERVADAS.

- | getPalavrasReservadas()
 - | Retorna as palavras reservadas cadastradas.
 - | Usa Collections.unmodifiableSet para impedir alteração externa do conjunto.

6. Classes e métodos de Clone.java

A classe Clone é a janela principal da IDE. Ela estende JFrame, que é a classe do Swing usada para criar janelas. Esse arquivo concentra a interface visual e as ações do usuário.

Atributos principais

- | Atributo | Função

- | editorArea | Área principal onde o usuário escreve o código.

- | lineNumberPanel | Painel lateral que desenha os números das linhas do editor.

- | statusLabel, posLabel, fileLabel | Labels da barra de status, posição do cursor e arquivo atual.

- | tokenTableModel, errorTableModel | Modelos de dados das tabelas de tokens e erros.

- | ultimosTokens, ultimosErros | Guardam o resultado mais recente da análise.

- | currentFile, modified | Controlam o arquivo aberto e se há alterações não salvas.

- | undoManager | Controla desfazer e refazer no editor.

- | currentTheme | Tema visual aplicado na interface.

| aplicandoDestaque | Evita loop ao aplicar destaque de sintaxe no documento.

Classes internas

| Classe interna | Responsabilidade

| Theme | Guarda as cores de um tema: fundo, painel, texto, borda, palavras-chave, numeros, strings e comentarios.

| ClassePillRenderer | Renderizador personalizado para exibir a classe do token com aparencia de etiqueta colorida na tabela.

| LineNumberPanel | Painel que calcula e desenha os numeros das linhas ao lado do editor.

Metodos de construcao da interface

| Metodo | O que faz

| Clone() | Construtor da janela. Define titulo, tamanho, comportamento de fechamento, tema inicial, monta a interface e registra eventos.

| buildUI() | Organiza a estrutura principal da janela: barra de titulo, area de trabalho e barra de status.

| buildTitleBar() | Cria a barra superior com titulo, menu e controles principais.

| buildMenuBar() | Cria menus como Arquivo, Editar, Exibir e Ajuda, ligando cada item a uma acao.

| menu(), menuItem() | Metodos auxiliares para criar menus e itens de menu com estilo padrao.

| styleMenu(), styleMenuItem() | Aplicam fonte e aparencia consistente aos menus.

| buildWorkspace() | Monta a area central com barra lateral, editor e resultados.

| buildSidebar() | Cria botoes laterais para funcoes rapidas, como novo, abrir, salvar e analisar.

| sidebarBtn(), sidebarSep() | Criam botoes e separadores reutilizaveis para a barra lateral.

| buildEditorSection() | Monta o bloco do editor, incluindo aba, barra compacta e painel de texto.

| buildEditorTabBar() | Mostra a area visual de aba do arquivo aberto.

| buildCompactToolbar() | Cria botoes de execucao, exportacao, fonte, tema e IA.

| tbBtn(), tbDiv() | Auxiliares para botoes e divisores da barra de ferramentas.

| buildEditorPane() | Cria o JTextPane, adiciona numeracao de linha, scroll e listeners de mudanca.

| buildResultsSection() | Monta a regio inferior/lateral onde aparecem tokens e erros.

| buildResultsTabs() | Cria abas para tabela de tokens e tabela de erros, com campos de pesquisa.

| buildStyledTable() | Cria uma tabela Swing com estilos padronizados e sem edicao direta.

| buildStatusBar() | Cria a barra inferior de status com informacoes de arquivo, cursor, tokens e erros.

Metodos de filtragem, navegacao e resultados

| Metodo | O que faz

| filtrarTokens() | Filtra a tabela de tokens de acordo com o texto digitado no campo de pesquisa.

| filtrarErros() | Filtra a tabela de erros usando o texto de busca.

| irParaErroSelecionado() | Quando o usuario seleciona um erro, tenta mover o cursor para a linha correspondente.

| extrairElrParaLinhaDoErro(String erro) | Extrai o numero da linha da mensagem de erro e chama irParaLinha().

| irParaLinha(int linha) | Calcula a posicao inicial da linha no documento e posiciona o cursor do editor.

| atualizarResultados() | Preenche as tabelas com os tokens e erros da ultima analise.

| limparResultados() | Remove linhas das tabelas e zera contadores quando nao ha analise valida.

Metodos de arquivo e acoes do usuario

| Metodo | O que faz

| acaoNovo() | Cria um novo documento, depois de confirmar descarte de alteracoes se necessario.

| acaoAbrir() | Abre um arquivo escolhido pelo usuario e carrega seu conteudo no editor.

| acaoSalvar() | Salva no arquivo atual. Se nao houver arquivo, chama acaoSalvarComo().

| `acaoSalvarComo()` | Abre seletor de arquivo para o usuario escolher onde salvar.

| `acaoSair()` | Fecha a aplicacao apos confirmar se existem mudancas nao salvas.

| `buildFileChooser()` | Configura o seletor de arquivos usado para abrir e salvar.

| `acaoAnalisar()` | Pega o texto do editor, cria um Lexer, executa `analisar()`, guarda tokens e erros e atualiza a interface.

| `acaoCarregarExemplo()` | Insere no editor um exemplo de codigo Mini Pascal para teste rapido.

| `acaoExportarResultado()` | Exporta o resultado da analise para arquivo, usando o texto produzido por `gerarRelatorio()`.

| `gerarRelatorio()` | Monta o conteudo textual/CSV com tokens e erros da ultima analise.

| `csv(String s)` | Prepara texto para ser salvo em formato CSV, tratando aspas e separadores.

Metodos de aparencia, fonte e destaque

| Metodo | O que faz

| `resolveFont(String name, int size)` | Escolhe uma fonte monoespacada disponivel ou usa alternativa padrao.

| `updateEditorFont()` | Aplica nome e tamanho da fonte atual no editor e atualiza a numeracao.

| `changeFontSize(int delta)` | Aumenta ou diminui o tamanho da fonte.

| `dialogFontSize()` | Abre uma janela para informar tamanho da fonte manualmente.

| `applyTheme(Theme t)` | Aplica cores do tema no editor, paines, tabelas e componentes da interface.

| `applyThemeToTable(JTable table, Theme t)` | Aplica cores e selecao visual em uma tabela especifica.

| `destacarPalavrasReservadas()` | Aplica coloracao no editor para palavras reservadas, numeros, strings e comentarios.

| `attr(Color fg, boolean bold, boolean italic)` | Cria um conjunto de atributos de texto para destaque.

| `highlight(...)` | Procura padroes por expressao regular e aplica estilo no trecho encontrado.

Metodos de estado e utilidade

| Metodo | O que faz

| onChange() | Executado quando o texto muda. Marca o arquivo como modificado, atualiza titulo, posicao e destaque.

| updatePosition() | Calcula linha e coluna atuais do cursor e mostra na barra de status.

| updateTitle() | Atualiza o titulo da janela, incluindo marcador de modificacao quando o arquivo nao foi salvo.

| setStatus(String msg) | Mostra uma mensagem curta na barra de status.

| confirmarDescarte() | Pergunta ao usuario se deseja descartar alteracoes nao salvas.

| erro(String msg) | Mostra uma caixa de mensagem de erro.

| main(String[] args) | Ponto de entrada do programa. Inicializa a interface Swing.

Metodos de IA local

| Metodo | O que faz

| sugerirCorrecaoComIA()

| Cria um prompt com o codigo e os erros encontrados, chama o modelo local e mostra sugestoes ao usuario.

| chamarLLM(String prompt)

| Envia uma requisicao HTTP para <http://localhost:11434/api/generate>, usando o modelo codellama:7b. Depende do Ollama estar rodando localmente.

7. Como a implementacao foi pensada

A implementacao separa a logica da analise lexica da interface grafica. Essa divisao e importante porque

Lexer pode ser estudado e testado sem depender do Swing, enquanto Clone fica responsavel por mostrar dados e receber a interacao do usuario.

O Lexer usa uma tecnica comum em compiladores simples: leitura sequencial com um indice.

Em vez de tentar interpretar o programa completo, ele olha apenas o caractere atual e, quando necessario,

olha o proximo caractere com proximoEh(). Isso permite reconhecer simbolos compostos como

:=, <= e ...

A interface foi implementada com Swing porque ele ja oferece componentes prontos, como JFrame,

JTextPane, JTable, JMenuBar, JScrollPane e

JFileChooser. O codigo personaliza esses componentes para parecer uma IDE: editor

com fonte
monoespacada, numeracao de linhas, temas escuros, barra lateral e tabelas de
resultados.

8. Exemplo de entrada e saida

Codigo de entrada:

```
program Escola;  
var  
idade: integer;  
begin  
idade := 18;  
end.
```

Tokens esperados:

| Lexema | Classe | Descricao

| program | PALAVRA_RESERVADA | Palavra com significado fixo.

| Escola | IDENTIFICADOR | Nome criado pelo programador.

| idade | IDENTIFICADOR | Nome criado pelo programador.

| integer | PALAVRA_RESERVADA | Tipo inteiro da linguagem.

| := | ATRIBUICAO | Atribuicao de valor.

| 18 | NUMERO_INTEIRO | Numero sem parte decimal.

| EOF | FIM_ARQUIVO | Marcador de fim da entrada.

9. Pontos fortes e limitacoes

| Pontos fortes | Limitacoes

| Codigo dividido em modelo, analisador e interface.

| A analise e apenas lexica; nao valida gramatica completa do Pascal.

| Reconhece palavras reservadas, identificadores, numeros, strings, comentarios, operadores e
separadores.

| Strings usam regra simples e nao tratam todos os escapes possiveis.

| Mostra erros com linha e permite navegar ate a linha do erro.

| A sugestao por IA depende de Ollama local e do modelo instalado.

| Interface facilita estudo visual dos tokens.

| O arquivo Clone.java concentra muitas responsabilidades e poderia ser dividido em classes menores.

10. Conclusao

O projeto e uma boa base didatica para entender a primeira fase de um compilador.

A classe Lexer

mostra como transformar texto bruto em tokens classificados. A classe Token

organiza os dados de

cada unidade reconhecida. A classe Clone transforma tudo isso em uma ferramenta

visual, permitindo

escrever codigo, executar a analise, ver resultados, filtrar informacoes, exportar

relatorios e estudar erros.

Para evoluir o projeto, o proximo passo natural seria criar um analisador

sintatico, responsavel por verificar

se a ordem dos tokens forma comandos validos segundo a gramatica da linguagem.